

수식	변수 의미	적용 조건	사용처
$T = IC \times CPI \times CT$	IC=실행 명령어 수, CPI=명령어당 사이클, $CT=1/ClockFreq$	CT는 HW 고정. 컴파일러는 IC,CPI만 줄일 수 있음	최적화 효과 평가
$OUT[b] = GEN[b] \cup (IN[b] - KILL[b])$	GEN=BB b의 locally generated defs, KILL=같은 변수의 다른 정의들, IN=b 진입 시 도달 정의	Forward 분석. KILL은 정적 속성 (IN에 있는 것만 실제 kill됨)	RD
$IN[b] = \cup OUT[p], \forall pred p$	$OUT[p]$ =선행자 p의 출구 도달 정의	합류점에서 union (어떤 경로든 도달하면 포함)	RD
$IN[b] = USE[b] \cup (OUT[b] - DEF[b])$	USE=locally exposed uses, DEF=BB b에서 정의된 변수, OUT=b 출구 시 live 변수	Backward 분석. DEF는 BB 내 정의된 모든 변수	LV
$OUT[b] = \cup IN[s], \forall succ s$	$IN[s]$ =후계자 s의 진입 live 변수	합류점에서 union (어떤 후속 경로든 사용되면 live)	LV
$Speedup = T_{old} / T_{new}$	T_{old} =최적화 전, T_{new} =최적화 후	$Speedup > 1$ 이면 개선	최적화 비교
$x \leq y \iff x \wedge y = x$	x가 y보다 격자에서 아래(또는 같은 위치)	$\wedge = \cup$ 이면 $y \subseteq x$, $\wedge = \cap$ 이면 $x \subseteq y$	반순서
단조: $x \leq y \rightarrow f(x) \leq f(y)$	입력이 내려가면 출력도 내려감	수렴 보장 핵심. $f(x \wedge y) \leq f(x) \wedge f(y)$ 와 동치	프레임워크
분배: $f(x \wedge y) = f(x) \wedge f(y)$	합치고적용 = 적용 하고합치기	성립 $\rightarrow MFP = MOP$. RD, LV는 분배적	프레임워크
$FP \leq MFP \leq MOP \leq Perfect$	해의 계층	분배적 $\rightarrow MFP = MOP$, 단조 $\rightarrow MFP \leq MOP$	정밀도
$rPO(i) = N - PostOrder(i)$	DFS post-order 뒤집기	Forward 분석 최적 방문 순서. 평균 ~2.75회 수렴	속도

프로그램 실행 방식 비교

항목	컴파일러	인터프리터 (VM)	하이브리드
대표 언어	C/C++	Python, JS	Java
실행 흐름	소스 $\rightarrow$ 기계어 $\rightarrow$ HW 실행	소스 $\rightarrow$ SW가 직접 해석 실행	소스 $\rightarrow$ 바이트코드 $\rightarrow$ VM(인터프리터/JIT)
성능	가장 빠름	느림	중간 (JIT으로 개선)
이식성	낮음 (타겟 종속)	높음	높음 (VM만 있으면 됨)
compile/runtime	분리	runtime만	2단계 (compile+runtime)

Reaching Definitions vs Live Variables vs Available Expr vs Anticipated Expr

항목	Reaching Def (RD)	Live Variables (LV)	Available Expr (AE)	Anticipated Expr (AntE)
질문	어떤 정의가 도달?	변수가 앞으로 사용?	식이 이미 계산됨?	식이 이미 사용됨?
도메인	정의 집합	변수 집합	표현식 집합	표현식 집합
방향	Forward	Backward	Forward	Backward
meet($\wedge$)	$\cup$ (어떤 경로든)	$\cup$ (어떤 경로든)	$\cap$ (모든 경로에서)	$\cap$ (모든 경로에서)
전달함수	$GEN \cup (IN - KILL)$	$USE \cup (OUT - DEF)$	$GEN \cup (IN - KILL)$	$USE \cup (OUT - DEF)$
GEN	BB 내 마지막 정의	locally exposed uses	BB에서 계산된 표현식	BB에서 사용된 표현식
KILL	같은 변수의 다른 정의	BB에서 정의된 변수	타겟변수 포함 표현식	타겟변수 포함 표현식
합류	$IN = \cup OUT[pred]$	$OUT = \cup IN[succ]$	$IN = \cap OUT[pred]$	$OUT = \cap IN[succ]$
Top($\top$)	{ }	{ }	U(전체)	U(전체)
초기화	$OUT[b] = \{ \}$	$IN[b] = \{ \}$	$OUT[b] = U$	$IN[b] = U$
경계	$IN[entry] = \{ \}$	$IN[exit] = \{ \}$	$IN[entry] = \{ \}$	$IN[exit] = \{ \}$
분배적	O (MFP=MOP)	O	O	O
용도	live range, 상수 전파	RA, DCE	Global CSE, LICM	PIR, DCE
안전 방향	더 많이 포함	더 많이 포함	더 적게 포함	더 많이 포함

지역 최적화 기법 비교

기법	패턴	변환	효과
Load-to-Copy	store 직후 같은 위치 load	load $\rightarrow$ copy	메모리 접근 제거
Local CSE	BB 내 동일 식 중복	두 번째 삭제/재사용	연산 제거
Constant Folding	컴파일 타임 계산 가능	상수로 교체	연산 제거
Dead Code Elim	결과 미사용	명령어 삭제	코드 축소
Copy Propagation	copy 소스로 대체	사용처를 원본으로	copy dead화

루프 최적화 기법 비교

기법	조건	변환	효과
LICM	루프 불변식	루프 밖으로 이동	반복 횟수만큼 절감
Strength Reduction	유도변수 존재	곱셈 $\rightarrow$ 덧셈	비싼 연산 제거
IV Elimination	SR 후 원래 IV 불필요	새 IV로 조건 교체, 원래 제거	레지스터/코드 절약

Copy Elimination 비교

기법	원리	조건	부작용
Copy Propagation	사용처를 원본으로 교체	항상 가능	없음
Copy Coalescing	소스/타겟 LR 합쳐 같은 레지스터	LR이 간섭 안 해야 함	간선 증가→컬러링 어려움

Register Allocation: Chaitin vs Briggs 비교

항목	Chaitin (1982)	Briggs (Optimistic)
Simplify시 degree>=n 처리	즉시 spill 후보 결정 + 제거	일단 스택에 push (미확정)
Spill 결정 시점	Simplify 단계	Select 단계
Spill 후 동작	load/store 삽입 → 처음부터 재시작	색 배정 불가 시에만 spill → 재시작
불필요한 spill	발생 가능	제거 (Chaitin spill 노드의 부분집합만 spill)
컬러링 능력	기본	Chaitin이 칠하는 모든 그래프 + 추가 그래프

데이터 흐름 분석 프레임워크 성질 비교

분석	meet($\wedge$)	방향	단조?	분배?	MFP vs MOP
Reaching Definitions	$\cup$ (union)	Forward	O	O	MFP = MOP
Live Variables	$\cup$ (union)	Backward	O	O	MFP = MOP
Available Expressions	$\cap$ (intersection)	Forward	O	O	MFP = MOP
Constant Propagation	특수 meet	Forward	O	X	MFP < MOP

격자 방향 비교 ($\wedge = \cup$ vs $\wedge = \cap$)

항목	$\wedge = \cup$ (RD, LV)	$\wedge = \cap$ (Available Expr)
Top($\top$)	{ } (빈 집합)	전체 집합
Bottom($\perp$)	전체 집합	{ } (빈 집합)
$\leq$ 의미	$x \leq y \iff x \cup y = y \iff y \subseteq x$ (큰 집합이 아래)	$x \leq y \iff x \cap y = x \iff x \subseteq y$ (작은 집합이 아래)
$\geq$ 의미	$x \geq y \iff x \subseteq y$ (작은 집합이 위)	$x \geq y \iff y \subseteq x$ (큰 집합이 위)
주의	★ $\wedge = \cup$ 에서 $\geq$ 는 “작은 집합”! 일반 직관($\geq$ =더 큼)과 반대	$\wedge = \cap$ 에서는 일반 직관과 일치

초기값	$T = \{\}$	$T = \text{전체}$
반복 방향	집합이 커지면서 내려감	집합이 작아지면서 내려감

용어	정의
AST	파스 트리에서 불필요한 노드를 제거한 추상 구문 트리
Available Expression	지점 p에 도달하는 모든 경로에서 계산+피연산자 미재정의인 표현식
Basic Block (BB)	제어 흐름이 시작에 들어와 끝에 나가며 중간 분기 없는 연속 명령어
CFG	BB를 노드, 제어 흐름을 간선으로 하는 방향 그래프
Copy Coalescing	COPY 소스/타겟 LR이 비간섭이면 합쳐 같은 레지스터, COPY 삭제
CPI	명령어 하나당 평균 클럭 사이클 수
Dead Code	결과가 후속 명령어에서 사용되지 않는 명령어
Definition	변수에 값을 대입하는 명령어 (쓰기)
DEF[b]	BB b에서 정의된 변수 집합
Fixed Point	반복 계산에서 값이 변하지 않는 수렴 상태
GEN[b]	BB b의 locally generated definitions 집합
Induction Variable	루프 반복마다 일정하게 변하는 변수
Interference Graph	노드=LR, 간선=동시 live. 그래프 컬러링으로 RA
IR	소스와 타겟 사이의 중간 표현
KILL[b]	BB b 내 정의와 같은 변수의 다른 정의들 (정적 속성)
Live Range (Web)	정의+도달 가능한 사용의 집합. RA 단위
Live Variable	지점 p에서 어떤 경로에서 재정의 전 사용되는 변수
Locally Exposed Use	BB 내 사용인데 앞에 같은 변수 정의가 BB 내 없는 것
Locally Generated Def	BB 내 변수의 마지막 정의
Memory Live Range	같은 메모리 위치 접근하는 store/load 집합
ϕ -function	SSA 합류점에 삽입. 경로에 따라 값 선택 (실행 안 되는 표기법)
Pseudo Register	물리 RA 전 가상 레지스터
Reaching Definition	정의 d가 p에 도달= $d \rightarrow p$ 경로 존재 +kill 안 됨
Register Promotion	load/store $\rightarrow$ copy. singleton 변수만 가능
Spilling	RA 시 레지스터 부족 $\rightarrow$ 일부 web을 스택으로
SSA	각 변수가 전체에서 단 한 번만 정의되는 형태
Strength Reduction	곱셈 $\rightarrow$ 덧셈 교체. 유도변수에 적용 BB 통과 시 데이터 흐름 정보 변화 함

Transfer Function	수
USE[b]	BB b의 locally exposed uses 집합
Worklist Algorithm	변화 노드만 재계산. 고정점까지 수렴
Descending Chain	격자에서 $x_0 \geq x_1 \geq \dots \geq x_n$ 인 수열. 유한하면 수렴 보장
Distributive	$f(x \wedge y) = f(x) \wedge f(y)$. 반복 알고리즘이 MOP과 동일한 해를 줌
GLB (Greatest Lower Bound)	x, y 모두보다 $\leq$ 이면서 가장 큰 원소. $x \wedge y$ 가 유일한 glb
Height (격자 높이)	가장 긴 하강 체인의 길이. RD에서 정의 수와 같음
Lattice (격자)	모든 원소 쌍이 유일한 glb와 lub를 가지는 반순서 집합
MFP (Maximum Fixed Point)	반복 알고리즘이 수렴한 결과. 모든 고정점 중 가장 큰 것
Monotone (단조)	$x \leq y \rightarrow f(x) \leq f(y)$. 입력이 내려가면 출력도 내려감. 수렴 보장
MOP (Meet Over all Paths)	모든 가능한 경로에 대해 meet한 해. $MFP \leq MOP \leq \text{Perfect}$
NAC	Not A Constant. 상수 전파에서 상수가 아닌 값
Partial Ordering (반순서)	반사적+반대칭+추이적인 이항 관계
Reverse Post-Order	DFS post-order를 뒤집은 순서. Forward 분석 최적 방문 순서
Semi-Lattice (반격자)	Top 존재+유일한 glb 존재. meet 연산자의 4성질에서 도출
Top ($\top$)	$x \wedge \top = x$ 인 원소. 격자의 최상위. 초기 값으로 사용
Bottom ($\perp$)	$x \wedge \perp = \perp$ 인 원소. 격자의 최하위
UNDEF	상수 전파에서 아직 값이 정해지지 않은 상태 (Top)

RISC 명령어 레퍼런스 (HP PA-RISC 기반)

수업 예제에서 사용되는 명령어. 표기: CMD src, dst (소스 먼저, 목적지 나중).

데이터 이동 (Load / Store / Copy)

명령어	문법	동작
LDW	LDW offset(reg), dst	메모리[reg+offset] $\rightarrow$ dst 레지스터에 로드 (word)
LDWS	LDWS offset(reg), dst	메모리[reg+offset] $\rightarrow$ dst 레지스터에 로드 (word, short displacement)
LDWS,MA	LDWS,MA offset(reg), dst	메모리[reg] $\rightarrow$ dst 로드 후, reg += offset (Modify After: 포인터 자동 증가)

LDWX,S	LDWX,S idx(base), dst	메모리[base + idx×4] → dst (인덱 스 스케일 로드, ×4 = word 크기)
STW	STW src, offset(reg)	src 레지스터 → 메모 리[reg+offset]에 저 장 (word)
STWS	STWS src, offset(reg)	src 레지스터 → 메모 리[reg+offset]에 저 장 (word, short displacement)
STWM	STWM src, offset(reg)	src → 메모리[reg]에 저장 후, reg += offset (Modify: 포인 터 자동 증가)
LDI	LDI imm, dst	즉시값(immediate) → dst 레지스터에 로 드
LDO	LDO offset(reg), dst	reg + offset → dst (Load Offset: 레지 스터+상수 덧셈, 메모 리 접근 아님)
COPY	COPY src, dst	src 레지스터 → dst 레지스터 복사

산술/논리 연산

명령어	문법	동작
ADD	ADD src1, src2, dst	src1 + src2 → dst
MULTI	MULTI imm, src, dst	imm × src → dst (즉시값 곱셈)
SUB	SUB src1, src2, dst	src1 - src2 → dst
SUBI	SUBI imm, src, dst	imm - src → dst (즉시값에서 뺄셈)
SH2ADD	SH2ADD src, base, dst	(src << 2) + base → dst (src×4 + base, 배열 인덱싱용)

주소 계산 (32비트 상수 로드)

명령어	문법	동작
ADDILG	ADDILG LR'sym, reg, dst	reg + symbol의 상 위 21비트 → dst (Long displacement 상위)
ADDIL	ADDIL LR'sym, reg, dst	ADDILG와 동일 (표 기 변형)
LDO (하위)	LDO RR'sym(src), dst	src + symbol의 하 위 11비트 → dst (Long displacement 하위)

ADDILG + LDO 쌍으로 32비트 전역 주소 획득. SPARC의 sethi+or, MIPS의 lui+ori에 대응.

분기 / 비교

명령어	문법	동작
IF	IF reg1 < reg2 GOTO label	reg1 < reg2이면 label로 분기 (의사 명령어)
IFNOT	IFNOT reg1 < reg2 GOTO label	조건 불만족 시 label 로 분기 (의사 명령어)
COMB,<=	COMB,<=,N src1, src2, label	src1 ≤ src2이면 label로 분기 (Compare and Branch)
COMBF,<	COMBF,<,N src1, src2, label	src1 < src2가 거짓 이면 분기 (Compare and Branch False)
COMIB,<=	COMIB,<=,N imm, reg, label	imm ≤ reg이면 label로 분기 (즉시값 비교)
COMIBF,<	COMIBF,<,N imm, reg, label	imm < reg가 거짓이 면 분기

복합 명령어 (Compound Instructions)

명령어	문법	동작
ADDIB,<	ADDIB,< imm, reg, label	reg += imm → 결과 < 0이면 label로 분 기 (증가+비교+분기)
ADDIBF,<	ADDIBF,< imm, reg, label	reg += imm → 결과 < 0이 거짓이면 분기
LDWS,MA	LDWS,MA offset(reg), dst	로드 + 포인터 자동 증가 (위 데이터 이동 참조)
STWM	STWM src, offset(reg)	저장 + 포인터 자동 증가 (위 데이터 이동 참조)

기타

명령어	문법	동작
NOP	NOP	아무 동작 안 함 (분기 delay slot 채우기 등 에 사용)
LDWCOPY	LDWCOPY src, dst	최적화 표기: load- to-copy 변환 후의 copy (수업 예제 전 용)

특수 레지스터

레지스터

역할

r0	항상 0 (상수 레지스터)
r27 (GP)	Global Pointer — 전역 데이터 영역 기준 주소
r30 (SP)	Stack Pointer — 스택 영역 기준 주소
r31	범용 (예제에서 임시 레지스터로 사용)
200+	의사 레지스터 (pseudo register, 물리 할당 전)

N 접미사 (Nullification)

,N 접미사가 붙은 분기 명령어: 분기가 실행되지 않을 때 다음 명령어(delay slot)를 무효화(nullify). 분기가 실행되면 delay slot도 정상 실행. 파이프라인 최적화용.

Ch1. 컴파일러 구조와 최적화 개요

1-1. 프로그램 실행 3가지 방식

(A) 컴파일러:

[Source] → Compiler → [Executable] → HW → Output
compile time ———┐ ┌ runtime

(B) 인터프리터:

[Source] → Interpreter(SW) → Output
└── runtime ─┘

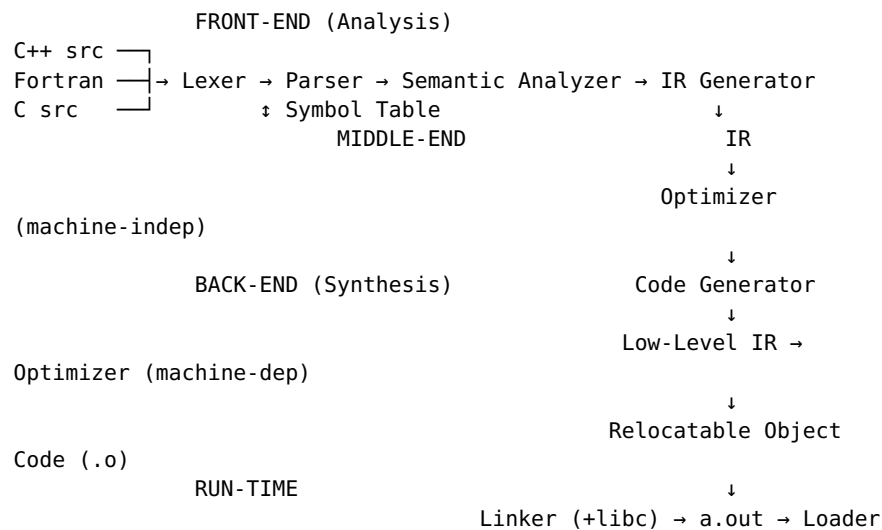
(C) 하이브리드:

[Source] → Compiler → [Bytecode IR] → VM(Interp/JIT) → Output
compile time ———┐ ┌ runtime ———

【예시문제】 Java 프로그램의 실행 과정을 단계별로 설명하라.

【풀이】: ① javac가 .java를 .class(바이트코드)로 컴파일 (compile time) → ② JVM이 바이트코드를 로드 → ③ 인터프리터로 실행하거나, 자주 실행되는 부분은 JIT 컴파일러가 기계어로 변환 후 HW에서 직접 실행 (runtime). 이식성: JVM만 있으면 어떤 CPU에서든 실행 가능.

1-2. 컴파일러 3단계 구조



→ HW

【예시문제】 “변수 x의 타입이 int인지 확인”은 어느 단계인가?

【풀이】: Front-end의 Semantic Analysis. 심볼 테이블에서 x의 선언을 찾아 타입을 확인. CFG로 표현 불가 → 자동 생성 불가, 직접 구현 필요.

1-3. LLVM 구조

C → Clang FE ↗ LLVM X86 BE → x86
Fortran → llvm-gcc FE ↗ LLVM Optimizer(LLVM IR) ↗ LLVM PPC BE →
PowerPC
Haskell → GHC FE ↗ LLVM ARM BE →
ARM

【예시문제】 LLVM에 M개 언어와 N개 아키텍처가 있을 때, 모듈 수는?

【풀이】: M(프론트엔드) + 1(공통 옵티마이저) + N(백엔드) = **M+N+1**개. 만약 단일 구조였으면 M×N개 필요. LLVM IR이 공통 인터페이스 역할.

1-4. Front-end: Lexer → Parser → Semantic Analyzer

입력: "int foo = 100;"

Lexer (정규식→유한오토마타):

→ [KW:int] [ID:foo] [OP:=] [NUM:100] [SYM:;]

Parser (CFG→파스트리):

```
var_decl
├─ type: "int"
├─ ID: "foo"
└─ init: NUM(100)
```

Semantic Analyzer (심볼테이블):

SymTable: {foo: type=int, scope=local, offset=SP-4}
check: 100은 int와 호환? → OK

【예시문제】 x = y + "hello";에서 Lexer, Parser, Semantic Analyzer 중 에러를 잡는 곳은?

【풀이】: Semantic Analyzer. Lexer는 토큰 분리만 (x, =, y, +, "hello", ; 모두 유효한 토큰). Parser는 expr = expr + expr 구조로 문법상 유효. Semantic Analysis에서 int와 string의 + 연산 타입 불일치를 검출.

1-5. 실행 시간 공식과 최적화

$T = IC \times CPI \times CT$

IC ← 컴파일러가 줄임 (CSE, DCE, 루프 최적화 등)
CPI ← 컴파일러가 줄임 (명령어 스케줄링, 레지스터 할당 등)
CT ← HW 결정 (컴파일러 관여 불가)

메모리 계층 지연:

L1 cache 0.5 ns ← 레지스터 할당으로 활용 극대화
Main memory 100 ns ← L1 대비 200배
Disk seek 10,000,000 ns ← L1 대비 2천만배

【예시문제】 IC=10⁹, CPI=1.5, Clock=2GHz. 최적화 후 IC 30%↓, CPI→1.2. Speedup은?

[풀이]:

$CT = 1/(2 \times 10^9) = 0.5ns$
 $T_{old} = 10^9 \times 1.5 \times 0.5ns = 0.75초$
 $T_{new} = 0.7 \times 10^9 \times 1.2 \times 0.5ns = 0.42초$
 $Speedup = 0.75/0.42 = 1.79배$

1-6. 최적화 컴파일러 페이지 파이프라인

- | | |
|---------------------------------|--------------------------|
| (1) Basic Block Optimization | ← BB 내 지역 최적화 |
| (2) Dataflow Analysis | ← RD, LV 등 정보 계산 |
| (3) Global CSE | ← BB 경계 넘어 중복 제거 |
| (4) Promotion of Memory Ops | ← load/store → 레지스터 |
| (5) LICM | ← 루프 불변 코드 이동 |
| (6) Induction Variable Analysis | ← 강도 감소 + IV 제거 |
| (7) Register Reassociation | |
| (8) Register Web Builder | ← Live Range 구축 |
| (9) Instruction Scheduling | ← pre-RA (병렬 실행) |
| (10) Register Allocation | ← 그래프 컬러링 |
| (11) Peephole Optimization | ← 패턴 기반 소규모 개선 |
| (12) Instruction Scheduling | ← post-RA (RA가 순서 변경하므로) |

[예시문제] Instruction Scheduling이 RA 전후 2번 수행되는 이유는?

[풀이]: pre-RA 스케줄링은 의사 레지스터(무한) 가정으로 자유롭게 재배치. RA가 물리 레지스터를 할당하면서 spill 코드 삽입/명령어 순서 변경이 발생할 수 있음. 따라서 post-RA에서 실제 레지스터 제약을 반영한 재스케줄링 필요.

1-7. 복합 명령어와 ISA 설계

```
for (i=0; i<N; i++) A[i] = 0;
```

최적화 후:

```
ADDIB,< 1,%r23,$003      ; i++ + (i<N?) + branch → 3동작 1명령어  
STWM  %r0,100(%r31)      ; store 0 + ptr+=100 → 2동작 1명령어
```

[예시문제] ADDIB,<가 수행하는 3가지 동작을 나열하라.

[풀이]: ① r23에 1 더함 (i++) ② 결과가 0보다 작은지 비교 (i<N, 카운터가 음수→양수) ③ 조건 만족 시 \$003으로 분기. 컴파일러 최적화가 이 패턴을 인식해 생성.

Ch2. 단계별 최적화 실전 예제

2-1. RISC 코드 생성: 스택머신 → RISC 변환

스택머신 IR (high-level):	RISC 의사코드 (low-level IR):
push → load/loadi	각 스택 슬롯에 의사 레지스터 부여
fetch → load	(s0, s1, s2, ...)
assign → store	스택 깊이(depth)를 추적하며 변환
add/sub/... → add/sub/...	

[예시문제] $x = y$ (x: 전역 GP+4, y: 지역 SP-4)의 스택코드를 RISC로 변환하라.

[풀이] (현재 스택 깊이=0으로 시작):

```
push_const GP+4 → loadi GP+4, s0      ; x주소 (depth:0→1)
```

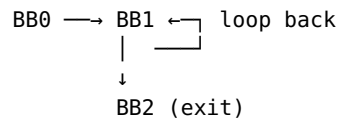
push_reg SP	→ copy SP, s1	; SP	(depth:1→2)
push_const 4	→ loadi 4, s2	; 4	(depth:2→3)
sub	→ sub s1, s2, s1	; SP-4	(depth:3→2)
fetch	→ load s1(0), s1	; y값	(depth:2→2)
assign	→ store s0(0), s1	; x=y	(depth:2→0)

2-2. 예제 프로그램과 비최적화 코드

```
int a[25][25]; // 전역. 행크기=25*4=100byte
main() { int i; for(i=0;i<25;i++) a[i][0]=0; }
```

```
STW  0, -40(30)          ; i=0 (스택)
LDW  -40(30), 206         ; r206=i
LDI  25, 212             ; r212=25
IFNOT 206<212 GOTO $02
$03: LDW  -40(30), 206      ; 중복#1
      ADDILG LR'a, 27, 213 ; &a 상위21bit+GP
      LDO  RR'a(213), 208   ; &a 하위11bit
      MULTI 100, 206, 214   ; i*100 (곱셈!)
      ADD  208, 214, 215    ; &a[i][0]
      STWS 0, 0(215)       ; a[i][0]=0
      LDW  -40(30), 206    ; 중복#2
      LDO  1(206), 216     ; i+1
      STW  216, -40(30)    ; i=i+1
      LDW  -40(30), 206    ; 중복#3
      LDI  25, 212        ; 중복 상수
      IF  206<212 GOTO $03
$02:
```

루프 본체 13개 명령어. CFG:



2-3. 지역 최적화 (BB 내) — 각 기법별 다이어그램과 예시

Load-to-Copy:

변환 전:		변환 후:	
STW r5, @mem	↗	STW r5, @mem	
LDW @mem, r8	↖	COPY r5, r8	← 메모리 접근 제거

【예시문제】 STW 0, -40(30) 직후 LDW -40(30), 206에 적용하라.

【풀이】: 같은 주소 -40(SP)에 0을 저장 직후 로드 → COPY 0, 206. r0=0이므로 r206=0.

Local CSE:

변환 전:		변환 후:	
x = y + z	(1번째)	x = y + z	
...		...	
w = y + z	(2번째)	w = x	← 재사용

【예시문제】 BB 내에서 LDW -40(30), 206이 2번 나타나고 중간에 -40(30)이나 r206이 변경되지 않았다면?

【풀이】: 두 번째 LDW -40(30), 206 삭제. 첫 번째 결과(r206)가 여전히 유효.

Constant Folding + Propagation:

변환 전:	변환 후:	
x = 3 * 4	x = 12	← Constant Folding
y = x + 1	y = 13	← Constant Propagation + Folding

Dead Code Elimination:

변환 전:	변환 후:	
x = y + 1	(삭제)	← x가 아래에서 덮어쓰여짐
...	...	
x = 10	x = 10	

Copy Propagation:

변환 전:	변환 후:	
x = y	(x=y dead→삭제 가능)	
z = x + 100	z = y + 100	← x 대신 y 사용

[예시문제] 아래 BB에 가능한 모든 지역 최적화를 적용하라.

```
a = load @x      ; (1)
b = a + 1        ; (2)
store b, @y      ; (3)
c = load @y      ; (4)
d = a + 1        ; (5)
e = 3 * 4        ; (6)
```

[풀이]:

(1) a = load @x	→ 유지
(2) b = a + 1	→ 유지
(3) store b, @y	→ 유지
(4) c = b	→ Load-to-Copy: (3)직후 load
(5) d = b	→ Local CSE: a+1 = (2)의 결과 b
(6) e = 12	→ Constant Folding: 3*4=12

2-4. Global CSE — Available Expression

Available Expression 조건:

- 지점 p에 도달하는 "모든 경로"에서 식 e가 계산됨
- + 마지막 계산 이후 e의 피연산자가 재정의되지 않음
- p에서 e가 다시 나타나면 중복 → 삭제

[예시문제] BB0에서 r212=25를 계산. BB1 시작점에서 r212가 available한가?

[풀이]: BB1에 도달하는 경로: ①BB0→BB1 (첫 진입), ②BB1→BB1 (루프백). 루프 끝에서 r212가 재정의되지 않았다면 두 경로 모두에서 r212=25가 유효 → **available** → BB1 내 LDI 25,212 삭제 가능.

2-5. Register Promotion

조건:

- ① Memory Live Range 구축 (같은 주소의 store/load 집합)
- ② singleton 변수 (배열/포인터/구조체 불가)
- ③ 주소 유도(address derivation)로 같은 위치 판별

변환:

store → copy
load → 삭제 (이미 레지스터에 있음)

Aliasing 문제 (Register Promotion의 최대 장벽):

포인터를 통한 간접 접근이 있으면, 해당 포인터가 promotion 대상 변수를 가리킬 가능성을 배제할 수 없음
→ 보수적으로 매번 메모리에서 읽어야 함

예시:

```
int g; int *p;
for (...) {
    sum += g;    ← g를 레지스터에 캐시하고 싶지만
    *p = i;      ← p가 &g를 가리키면 g가 변함!
}
```

→ 컴파일러는 p→g aliasing을 배제 못함 → g promotion 불가
→ sum, i는 지역변수(주소 미노출) → promotion 가능

【예시문제】 다음 중 Register Promotion 가능한 것은?

#	코드	가능?	이유
1	STW r5, SP-16; LDW SP-16, r8	O	singleton 지역변수
2	STW r5, 0(r3) (r3=포인터)	X	포인터 — 주소가 변
3	STW r5, GP+arr+r2*4	X	배열 — 인덱스가 변
4	지역변수 g + 함수 내 *p = val 존재	X	aliasing — p가 &g 가리킬 가능성

2-6. 루프 최적화: LICM + Strength Reduction + IV Elimination

LICM:

변환 전:	변환 후:	
loop:	p = &A;	← 루프 밖으로
p = &A; ← 불변	loop:	
... use p ...	... use p ...	

Strength Reduction:

$i=0,1,\dots,N-1 \rightarrow i*K = 0,K,2K,\dots$
MULTI K,i,t → 초기 t=0; 루프 내 t=t+K (곱셈→덧셈)

IV Elimination:

SR 후 원래 i가 조건에만 사용 → 조건을 t 기반으로 교체 → i 제거

【예시문제】 아래 루프에 SR + IV Elimination을 적용하라.

```
for (j=0; j<50; j++)
    C[j*12] = D[j*6+3];
```

【풀이】:

Step 1 – SR: 새 유도변수 도입

t1=0 (j*12 대체), t2=3 (j*6+3 대체)
loop: C[t1]=D[t2]; t1+=12; t2+=6;

Step 2 – IV Elim: $j<50 \rightarrow t1<600 (=50*12)$

while(t1<600) { C[t1]=D[t2]; t1+=12; t2+=6; }
→ j 제거. 곱셈 2개→덧셈 2개.

2-7. Register Allocation — 그래프 컬러링

Interference Graph:

x	—	y	x, y 동시 live → 다른 색 필요
	/		
	/		K개 물리 레지스터로 K-coloring
z		w	NP-complete → 휴리스틱

Chaitin의 6단계 (1982):

1. Renumber: live range 이름 재부여
2. Build: 간접 그래프 구축
3. Coalesce: copy 관련 노드 중 간접 없는 쌍 합치기 (aggressive)
4. Spill cost: $c \times 10^d$ 계산
5. Simplify: degree < n인 노드 → 스택 push
degree >= n → 최저 cost/benefit 노드를 spill 후보로 결정

6. Select: 스택에서 pop하며 색 배정
→ spill 후보가 있었으면 load/store 삽입 후 1번부터 재시작

Chaitin과 차이점:
 Simplify: degree $\geq n$ 노드도 "일단 스택에 push" (spill 확정하지 않음)
 Select: 스택에서 pop할 때 이웃들이 이미 제거되어
 실제 색칠 가능할 수 있음 $\rightarrow$ 가능하면 색 배정
 불가능할 때만 비로소 spill 결정

- Chaitin이 spill하는 노드 중 일부를 spill 없이 색칠 가능
- 불필요한 spill (unproductive spill) 제거
- spill 필요 시에도 Chaitin이 spill할 노드의 부분집합만 spill

- 컬러링 실패 시 전체 live range를 spill하는 대신 live range를 부분으로 분할 → 일부만 spill, 나머지 레지스터 유지
- 간접 그래프가 단순해져서 컬러링 가능성 증가
- LLVM: greedy register allocator + aggressive live range splitting

$$\begin{array}{ccc} a & \text{---} & b \\ | & / & | \\ | & / & | \\ c & & d \end{array}$$

[풀이]: a-b 간섭, a-c 간섭, b-c 간섭, b-d 간섭. $a \rightarrow R1$, $b \rightarrow R2$, $c \rightarrow R3$, $d \rightarrow R1$ (또는 $R3$). a-d 간섭 없으므로 같은 색 가능. 4개 web을 3색으로 배정 가능 $\rightarrow$ spilling 불필요.

【예시문제】 위 그래프에서 레지스터가 2개뿐이라면? Chaitin과 Briggs 각각의 동작을 비교하라.

n=2. 모든 노드의 degree ≥ 2 인 경우:

Chaitin: degree < 2 인 노드 없음 $\rightarrow$ d(degree=1)부터 spill 후보 결정
 $\rightarrow$ d 제거 후 a,b,c 중 degree < 2 가 나타나는지 반복

Briggs: d를 일단 스택에 push $\rightarrow$ 나머지에서 degree < 2 발생 가능
 $\rightarrow$ select 시 d의 이웃 중 이미 같은 색이 아닌 경우 색칠 성공 가능

2-8. Copy Elimination

【예시문제】 아래에 (a) copy propagation, (b) copy coalescing을 적용하라.

```
COPY r1, r2
ADD  r2, r3, r4
MUL  r2, r5, r6
```

【풀이 (a)】: r2 사용을 r1으로 교체 $\rightarrow$ COPY dead $\rightarrow$ 삭제:

```
ADD  r1, r3, r4
MUL  r1, r5, r6
```

【풀이 (b)】: r1과 r2의 LRO가 간섭 안 하면 합쳐서 같은 물리 레지스터 $\rightarrow$ COPY rX, rX $\rightarrow$ 삭제. 주의: r1이 COPY 이후에도 live하면 간섭 $\rightarrow$ coalescing 불가.

2-9. SSA Form

변환 규칙:

- ① 같은 변수 재정의 $\rightarrow$ 새 이름 부여 ($a \rightarrow a_0, a_1, a_2, \dots$)
- ② 제어 합류점 $\rightarrow \phi$ -function 삽입
 $a_{merged} = \phi(a_{then}, a_{else})$
- ③ ϕ 는 실제 실행 안 되는 표기법

이점: 변수 이름만으로 어느 정의인지 즉시 식별
 $\rightarrow$ 상수 전파 등 분석/최적화 극적 단순화

【예시문제】 아래를 SSA로 변환하고 상수 전파를 적용하라.

```
x = 5; y = x+1; if(y>3) x = y*2; z = x+y;
```

【풀이】:

SSA 변환:

```
x0=5; y0=x0+1; if(y0>3) x1=y0*2;
x2= $\phi$ (x1,x0); z0=x2+y0;
```

상수 전파:

```
x0=5, y0=6, 6>3=true  $\rightarrow$  then만 실행
x1=12, x2= $\phi$ (12,5)=12, z0=12+6=18
```

2-10. 전체 최적화 추적 요약

[원본] 루프 13개 명령어 (LDW $\times$ 4, LDI $\times$ 2, MULTI, ADDILG+LDO 매반복)

- $\downarrow$ (1) BB Opt: load-copy, CSE $\rightarrow$ 중복 LDW 일부 제거
- $\downarrow$ (2) Global CSE: BB 넘어 LDW, LDI 추가 삭제
- $\downarrow$ (3) Reg Promotion: i의 모든 STW/LDW $\rightarrow$ COPY/삭제
- $\downarrow$ (4) Loop Opt: ADDILG/LDO 루프밖(LICM), MULTI $\rightarrow$ LDO(SR), i제거(IVE)
- $\downarrow$ (5) Live Range: dead code 삭제 (COPY 0,206 + LDI 25,212)
- $\downarrow$ (6) Scheduling: 독립 명령어 재배치 (2-ALU)
- $\downarrow$ (7) RA + Copy Elim: COPY propagation/coalescing $\rightarrow$ 삭제

[최종] 루프 4개 명령어:

ADD 65,69,68 / STWS 0,0(68) / LD0 100(69),69 / IF 69<71 GOTO

Ch5. Global Common Subexpression Elimination

5-1. Available Expression Analysis

정의: 표현식 $x+y$ 가 지점 p 에서 available하다

⇒ p 까지의 모든 경로에서

- ① $x+y$ 가 최소 한 번 계산되었고
- ② 마지막 계산 이후 x 나 y 가 재정의되지 않음

분석 특성:

방향: Forward

합류 연산자: $\cap$ (intersection) – must 분석

→ 한 경로에서라도 available 아니면 전체 불가

초기값: $IN[entry] = \{\}$ (아무것도 available 아님)

$IN[나머지] = U$ (전체 표현식, optimistic)

전달합수: $OUT[b] = GEN[b] \cup (IN[b] - KILL[b])$

$GEN[b] = b$ 에서 계산되고 이후 피연산자 재정의 없는 표현식

$KILL[b] = b$ 에서 재정의되는 변수를 포함하는 모든 표현식

CSE 제거 절차:

1. Available Expression 분석으로 각 BB 입구의 available set 확보
2. BB 내부에서 value numbering (available로 초기화)
3. 이미 available한 표현식이 다시 나오면 → 이전 계산 결과 재사용 (같은 RHS → 같은 타겟 레지스터 가정)

[예시문제] 아래 CFG에서 B4 입구의 available expressions를 구하라.

```
      [B1] t1 = a+b
      /      \
[B2]          [B3]
a = 3      t2 = a+b
  \      /
      [B4] t3 = a+b
```

[풀이]: B2 경로: B1에서 $a+b$ 계산 → B2에서 a 재정의 → $a+b$ 무효화. B3 경로: B1, B3 모두에서 $a+b$ 계산, 재정의 없음 → available. 합류: $available(B2 \text{ 경로}) \cap available(B3 \text{ 경로}) = \{\} \cap \{a+b\} = \{\}$. B4 입구에서 $a+b$ 는 **available하지 않음**.

5-2. Partial Redundancy Elimination (PRE)

PRE = 부분적으로 중복인 계산을 완전히 중복으로 만들어 제거

일부 경로에서만 available → 그 경로에 계산을 삽입

→ 모든 경로에서 available로 만든 뒤 CSE로 제거

주의: 아무 곳이나 삽입하면 오류 발생

예: if (a>0) { b=1; d=b+c; } e=b+c;

잘못: if문 앞에 $t=b+c$ 삽입 → $a \leq 0$ 일 때 b 미정의 상태에서 계산!

5-3. Anticipation Analysis

정의: 표현식 E 가 지점 P 에서 anticipated(예상)된다

⇒ P 에서 시작하는 모든 실행 경로에서

같은 값을 계산하는 E가 반드시 나타남

용도: PRE에서 "E를 P에 삽입해도 되는가?"의 필요 조건
(anticipated해야만 삽입 후보, 단 충분 조건은 아님)

분석 특성:

방향: Backward (미래 경로에서 나타나는지 확인)

합류 연산자: $\cap$ (intersection) – 모든 경로에서 나타나야 함

전달함수:

$w = x + y \rightarrow x+y$ 를 GEN (블록이 $x+y$ 를 계산함)

$x = x + y \rightarrow x+y$ 를 KILL (피연산자 x 재정의로 이전 값 무효)

Ch3. Data Flow Analysis

3-1. 데이터 흐름 분석의 목적과 2단계 전략

목적: 함수의 각 지점에서 데이터 조작 정보 제공
→ 최적화의 기반 (분석 없이 최적화 불가)

1단계 Global: CFG의 BB 단위 → BB 경계 정보

BB 효과를 요약(GEN/KILL 등) → 반복 계산 → 고정점

2단계 Local: BB 내부 → 명령어 경계 정보

전역 결과를 출발점으로 명령어 효과 순차 적용

【예시문제】 왜 BB 단위로 나눠 분석하는가?

【풀이】 BB 내부에는 분기가 없어 선형 분석으로 충분. 복잡한 제어 흐름(분기/합류/루프)은 전역 분석에서 BB 단위로 처리. 2단계 분리로 효율적.

3-2. 명령어와 BB의 효과

명령어 $a = b + c$:

USE: {b, c} (읽기)

KILL: a의 이전 정의 (덮어쓰기)

GEN: a의 새 정의 (생성)

BB 효과 (명령어 합성):

Locally Exposed Use: 사용인데 앞에 같은 변수 정의가 BB 내 없음

Locally Gen Def: 변수의 BB 내 마지막 정의

Kill: BB 내 정의가 같은 변수의 BB 외부 정의를 무효화

【예시문제】 아래 BB의 exposed uses와 generated defs를 구하라.

```
r4 = r1 + r2
r2 = r4 + 1
r5 = r3 + r1
r1 = r5
r4 = r2 * r4
r2 = r5
```

【풀이】 (각 줄에서 exposed/not exposed 판별):

```
r4 = r1 + r2    ; r1/exposed, r2/exposed (둘 다 BB 내 첫 사용)
r2 = r4 + 1     ; r4: NOT exposed (위에서 정의)
r5 = r3 + r1    ; r3/exposed, r1: exposed (위에서 정의 안 됨→여전히 exposed)
r1 = r5         ; r5: NOT exposed
r4 = r2 * r4    ; r2, r4: NOT exposed
r2 = r5         ; r5: NOT exposed
```

Exposed Uses = {r1, r2, r3}
 Gen Defs = {r1(=r5 최종), r4(=r2*r4 최종), r2(=r5 최종)}

3-3. Reaching Definitions — 전달함수와 알고리즘

전달함수 (Forward):

$$\text{OUT}[b] = \text{GEN}[b] \cup (\text{IN}[b] - \text{KILL}[b])$$

↑
나가는
정의

↑
BB에서
생성된 정의

↑
들어온 것 중
kill 안 된 것

합류: $\text{IN}[b] = \cup \text{OUT}[\text{pred}]$ (union: 어떤 경로든 도달하면 포함)

경계: $\text{OUT}[\text{entry}] = \{\}$

초기: $\text{OUT}[b] = \{\}$

KILL의 의미 (주의!):

KILL = 정적 속성 (static property)

"같은 변수를 정의하는 함수 내 다른 명령어 목록"

실제 kill 효과: IN에 포함된 것만 영향받음

예: $\text{IN}=\{\} \rightarrow \{\} - \text{KILL} = \{\} \rightarrow \text{KILL}$ 효과 없음

GEN/KILL 계산 절차:

Step 1: 함수 전체의 변수별 정의 위치를 정리

변수 x: d1(B1), d4(B2), d7(B4)

변수 y: d2(B1), d5(B2)

Step 2: 각 BB에 대해

$\text{GEN}[b]$ = BB 내 각 변수의 마지막 정의 번호 수집

$\text{KILL}[b]$ = BB 내 정의된 각 변수에 대해,

함수 전체에서 같은 변수를 정의하는 "다른" 명령어 번호 수집

[예시문제] 아래 CFG의 RD를 구하라.

entry $\rightarrow$ B1 $\rightarrow$ B2 $\leftarrow$ B3(루프백)

↓ ↘
B3 B4 $\rightarrow$ exit

B1: d1:i=m-1, d2:j=n, d3:a=u1

B2: d4:i=i+1, d5:j=j-1

B3: d6:a=u2

B4: d7:i=u3

[풀이 Step 1] 변수별 정의: $i \rightarrow \{d1, d4, d7\}$, $j \rightarrow \{d2, d5\}$, $a \rightarrow \{d3, d6\}$

BB	GEN	KILL
B1	{1,2,3}	$i \rightarrow \{4,7\}, j \rightarrow \{5\}, a \rightarrow \{6\} = \{4,5,6,7\}$
B2	{4,5}	$i \rightarrow \{1,7\}, j \rightarrow \{2\} = \{1,2,7\}$
B3	{6}	$a \rightarrow \{3\} = \{3\}$
B4	{7}	$i \rightarrow \{1,4\} = \{1,4\}$

[풀이 Step 2] 반복 (초기 OUT={}):

회	BB	IN	OUT	변화
1	B1	{}	{1,2,3}	Y
1	B2	$\{1,2,3\} \cup \{\} = \{1,2,3\}$	$\{4,5\} \cup \{3\} = \{3,4,5\}$	Y

1	B3	{3,4,5}	{6} ∪ {4,5} = {4,5,6}	Y
1	B4	{3,4,5}	{7} ∪ {3,5} = {3,5,7}	Y
2	B2	{1,2,3} ∪ {4,5,6} = {1..6}	{4,5} ∪ {3,4,5,6} = {3,4,5,6}	Y
2	B3	{3,4,5,6}	{4,5,6}	N
2	B4	{3,4,5,6}	{3,5,6,7}	Y
3	B2	{1,2,3} ∪ {4,5,6} = {1..6}	{3,4,5,6}	N→수 렴

[최종]: B1:IN={},OUT={1,2,3} / B2:IN={1..6},OUT={3,4,5,6} /
B3:IN={3,4,5,6},OUT={4,5,6} / B4:IN={3,4,5,6},OUT={3,5,6,7}

3-4. Live Variables — 전달함수와 알고리즘

전달함수 (Backward):

IN[b] = USE[b] ∪ (OUT[b] - DEF[b])

↑ ↑ ↑
 진입 시 BB에서 나간 live 변수 중
 live 사용된 변수 BB에서 재정의 안 된 것

합류: OUT[b] = ∪ IN[succ] (union: 후속 경로 중 하나라도 사용하면 live)

경계: IN[exit] = {}

초기: IN[b] = {}

변화 시: predecessor를 ChangeNodes에 추가 (← RD와 반대!)

USE/DEF 계산 절차:

USE[b]: BB를 위에서 아래로 읽으며, 각 변수의 첫 접근이
사용(read)이면 USE에 추가.

정의(write)가 먼저 나오면 USE에 포함 안 됨.

DEF[b]: BB 내에서 정의된 모든 변수.

[예시문제] 같은 CFG에서 LV를 구하라.

[풀이 Step 1] USE/DEF:

BB	코드	USE	DEF
B1	i=m-1,j=n,a=u1	{m,n,u1}	{i,j,a}
B2	i=i+1,j=j-1	{i,j} (정의 전 사용)	{i,j}
B3	a=u2	{u2}	{a}
B4	i=u3	{u3}	{i}

[풀이 Step 2] 반복 Backward (초기 IN={}):

회	BB	OUT	IN	변화
1	B4	{}	{u3}	Y
1	B3	IN[B2]={}	{u2}	Y
1	B2	IN[B3] ∪ IN[B4] = {u2,u3}	{i,j} ∪ {u2,u3} = {i,j,u2,u3}	Y
1	B1	IN[B2] = {i,j,u2,u3}	{m,n,u1} ∪ {u2,u3} = {m,n,u1,u2,u3}	Y
2	B3	IN[B2] = {i,j,u2,u3}	{u2} ∪ {i,j,u2,u3} = {i,j,u2,u3}	Y
2	B2	{i,j,u2,u3} ∪ {u3} = {i,j,u2,u3}	{i,j,u2,u3}	N→수 렴

[최종]: B1:OUT={i,j,u2,u3},IN={m,n,u1,u2,u3} / B2:OUT={i,j,u2,u3},IN={i,j,u2,u3} / B3:OUT={i,j,u2,u3},IN={i,j,u2,u3} / B4:OUT={},IN={u3}

3-5. Local Analysis (BB 내부 명령어 경계)

전역 분석 결과(BB 시작 정보)로부터 명령어 하나씩 효과 적용:

```
IN(BB) = {d1, d2}          ← 전역 분석에서 제공
↓
x = 1 (d3) → kill d1 → {d2, d3}
↓
y = 1 (d4) → kill d2 → {d3, d4}
↓
z = x+y (d5) →           {d3, d4, d5}
```

[예시문제] BB 진입 시 RD IN={d1(x=0), d2(y=0), d5(z=?)} . 아래 각 명령어 후 도달 정의를 구하라.

```
x = 1      (d3)
y = x+1    (d4)
```

[풀이]:

```
시작:           {d1, d2, d5}
d3: x=1 → kill d1(x의 이전 정의) → gen d3 → {d2, d3, d5}
d4: y=x+1 → kill d2(y의 이전 정의) → gen d4 → {d3, d4, d5}
```

3-6. 보수적 근사 (Conservative Approximation)

원칙: 정확한 정보 계산 불가 시 → 안전한 방향으로 근사
 RD: 실제보다 더 많은 정의가 도달한다고 가정 (union)
 → 최적화 기회를 놓칠 수 있지만 잘못된 최적화는 방지
 LV: 실제보다 더 많은 변수가 live라고 가정 (union)
 → 레지스터를 더 쓸 수 있지만 live 변수를 죽이지는 않음

union을 쓰는 이유: "어떤 경로든 가능하면 포함" → 안전
 intersection을 쓰면: "모든 경로에서" → Available Expression에 사용

[예시문제] RD에서 합류에 intersection을 쓰면 어떤 문제가 생기나?

[풀이]: 정의 d가 경로 A로만 도달하고 경로 B로는 도달하지 않으면, intersection은 d를 제외. 하지만 런타임에 경로 A가 실행되면 d가 실제로 도달 → d를 무시하면 잘못된 최적화 (예: d가 도달 안 한다고 가정하고 다른 값을 사용). union이 안전.

Ch3 응용문제: Live Variables 전체 풀이

[문제] RD 예제와 같은 CFG에서 USE/DEF 계산 후 LV의 IN/OUT을 구하라.

[Step 1] USE/DEF 계산 — BB를 위에서 아래로 읽으며 각 변수의 첫 등장이 읽기인지 쓰기인지 판별:

```
B1:
d1: i = m-1
    오른쪽 m → m의 첫 등장이 읽기 → USE에 m 추가!
    왼쪽 i → DEF에 i 추가. i의 첫 등장이 쓰기 → USE에 i 안 넣음.
d2: j = n
    오른쪽 n → 첫 등장 읽기 → USE에 n! 왼쪽 j → DEF에 j.
d3: a = u1
```

오른쪽 $u1 \rightarrow$ 첫 등장 읽기 $\rightarrow$ USE에 $u1$! 왼쪽 $a \rightarrow$ DEF에 a .
 $\rightarrow$ USE={m,n,u1}, DEF={i,j,a}

B2:

d4: $i = i + 1$
 * 오른쪽 i (읽기)가 왼쪽 i (쓰기)보다 먼저 실행!
 i 의 첫등장이 읽기 $\rightarrow$ USE에 i 추가! 왼쪽 $i \rightarrow$ DEF에 i .
 d5: $j = j - 1$
 j 의 첫등장이 읽기 $\rightarrow$ USE에 j 추가! 왼쪽 $j \rightarrow$ DEF에 j .
 $\rightarrow$ USE={i,j}, DEF={i,j}
 (USE와 DEF에 같은 변수 가능! "정의도 하지만 사용이 먼저")

B3: d6: $a = u2 \rightarrow$ USE={u2}, DEF={a}

B4: d7: $i = u3 \rightarrow$ USE={u3}, DEF={i}

[Step 2] successor 관계: $B1 \rightarrow B2$, $B2 \rightarrow \{B3, B4\}$, $B3 \rightarrow B2$ (루프백),
 $B4 \rightarrow \text{exit}$

[Step 3] 반복 (Backward, exit쪽에서 entry쪽으로) 초기: 모든 $IN = \{\}$:

— 1회차 —

B4: $OUT = IN[\text{exit}] = \{\}$
 $IN = \{u3\} \cup (\{\} - \{i\}) = \{u3\}$
 해석: $u3$ 만 live. B4에서 $i=u3$ 할 때 $u3$ 를 읽어야 하니까. 변화!

B3: $OUT = IN[B2] = \{\}$ (B2의 IN 아직 $\{\}$)
 $IN = \{u2\} \cup (\{\} - \{a\}) = \{u2\}$
 해석: $u2$ 만 live. B3에서 $a=u2$ 할 때 $u2$ 를 읽어야 하니까. 변화!

B2: $OUT = IN[B3] \cup IN[B4] = \{u2\} \cup \{u3\} = \{u2, u3\}$
 $IN = \{i, j\} \cup (\{u2, u3\} - \{i, j\})$
 $= \{i, j\} \cup \{u2, u3\}$ $\leftarrow u2, u3$ 는 DEF={i,j}에 없어서 살
 아남음
 $= \{i, j, u2, u3\}$
 해석: i, j 는 B2 안에서 사용(USE). $u2, u3$ 는 B2를 통과해 전파. 변화!

B1: $OUT = IN[B2] = \{i, j, u2, u3\}$
 $IN = \{m, n, u1\} \cup (\{i, j, u2, u3\} - \{i, j, a\})$
 $= \{m, n, u1\} \cup \{u2, u3\}$ $\leftarrow i, j$ 는 DEF에 있어서 제거됨!
 $= \{m, n, u1, u2, u3\}$
 해석: $m, n, u1$ 은 B1 안에서 사용. $u2, u3$ 는 통과 전파. i, j 는 B1에서
 새로 만들므로($i=m-1$ 등) 진입 시 이전 값 불필요 $\rightarrow$ 제거. 변화!

— 2회차 (B2의 IN이 변했으므로 B3의 OUT이 영향받음) —

B3: $OUT = IN[B2] = \{i, j, u2, u3\}$ $\leftarrow$ 1회차에서는 $\{\}$ 였음!
 $IN = \{u2\} \cup (\{i, j, u2, u3\} - \{a\})$
 $= \{u2\} \cup \{i, j, u2, u3\}$ $\leftarrow a$ 는 DEF에 있지만 OUT에 없어서
 무관
 $= \{i, j, u2, u3\}$
 해석: $u2$ 는 B3에서 사용. $i, j, u3$ 는 B3에서 안 건드리므로
 OUT에서 통과. B2에서 i, j 를, B4에서 $u3$ 를 쓸 것이므로. 변화!

B2: $OUT = IN[B3] \cup IN[B4] = \{i, j, u2, u3\} \cup \{u3\} = \{i, j, u2, u3\}$ $\leftarrow$ 1회차
 와 동일!
 $IN = \{i, j, u2, u3\}$ 변화
 없음 $\times$

B1: OUT, IN 모두 변화 없음 $\times$

— 3회차: B3도 변화 없음 → 수렴! —

[최종 답]:

B1: OUT={i, j, u2, u3}	IN={m, n, u1, u2, u3}
B2: OUT={i, j, u2, u3}	IN={i, j, u2, u3}
B3: OUT={i, j, u2, u3}	IN={i, j, u2, u3}
B4: OUT={}	IN={u3}

결과 해석: - B2의 i,j: B2 내 i=i+1, j=j-1에서 오른쪽(읽기)에 사용 → live - B2의 u2,u3: B2가 안 건드려서 통과. B3(u2), B4(u3)에서 사용 예정 - B3의 i,j: B3(a=u2)에서 사용 안 하지만 OUT에서 통과(DEF={a}에 없음). B2에서 쓸 예정 - B1의 IN에서 i,j 빠짐: B1에서 i=m-1, j=n으로 새로 정의 → 이전 값 불필요 - B4의 OUT={}: exit 후 아무것도 안 쓰이므로

Ch4. Foundations of Data Flow Analysis — 수학적 기초

4-1. 통합 프레임워크: (V, $\wedge$, F)

모든 데이터 흐름 분석은 3가지로 정의:

V = 값의 도메인 (예: 정의 집합, 변수 집합)
 $\wedge$ = meet 연산자 (예: $\cup$, $\cap$)
F = 전달 함수 집합 (예: $\text{GEN}_U(x - \text{KILL})$)

같은 성질을 가진 문제들에 대해
수렴·정확성·정밀도·속도를 한번에 증명 가능!

[예시문제] RD의 (V, $\wedge$, F)는?

[풀이]: $V = \{x \mid x \subseteq \{d_1, \dots, d_n\}\}$, $\wedge = \cup$, $F: f(x) = \text{GEN} \cup (x - \text{KILL})$

4-2. Meet 연산자의 4가지 성질과 반순서

$\wedge$ 가 만족해야 하는 성질:

교환: $x \wedge y = y \wedge x$
멱등: $x \wedge x = x$
결합: $x \wedge (y \wedge z) = (x \wedge y) \wedge z$
Top: $x \wedge \top = x$

이 4성질로부터 반순서($\leq$)를 정의:

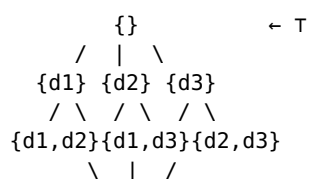
$x \leq y \Leftrightarrow x \wedge y = x$
"x가 y보다 격자에서 아래(또는 같은 위치)"

[예시문제] $\wedge = \cup$ 일 때, $\{d_1, d_2\} \leq \{d_1\}$ 인가?

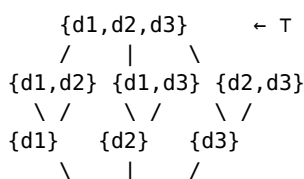
[풀이]: $\{d_1, d_2\} \wedge \{d_1\} = \{d_1, d_2\} \cup \{d_1\} = \{d_1, d_2\} = \text{왼쪽과 같음} \rightarrow$
YES. 큰 집합이 격자에서 아래.

4-3. 격자 다이어그램

$\wedge = \cup$ (RD)일 때:



$\wedge = \cap$ (Available Expr)일 때:



【예시문제】 단조성이 $f(x) \leq x$ 를 의미하는가?

[풀이]: 아니다! $GEN=\{d1\}$, $KILL=\{d2\}$, $x=\{d2\}$ 이면 $f(x)=\{d1\}$. $\{d1\}$ 과 $\{d2\}$ 는 비교 불가. 단조성은 “입력 간의 관계가 출력에서 보존된다”는 것이지 “출력이 항상 입력보다 작다”가 아님.

4-6. 분배성 — MFP = MOP

분배(distributive): $f(x \wedge y) = f(x) \wedge f(y)$ (등호!)

반복 알고리즘: 합치고 적용 = $f(x \wedge y)$

이상적 해(MOP): 적용하고 합치기 = $f(x) \wedge f(y)$

분배적 $\rightarrow$ 둘이 같음 $\rightarrow$ 정밀도 손실 없음!

RD, LV, Available Expr: 분배적 $\checkmark$

Constant Propagation: 비분배 $\times$ (정밀도 손실)

[예시문제] 상수 전파가 비분배인 이유를 예시로 보여라.

[풀이]:

경로 A: $x=2, y=3$ 경로 B: $x=3, y=2$

\ /
 $\rightarrow z = x + y \leftarrow$

적용 후 합치기: $f(A)=\{z=5\}$, $f(B)=\{z=5\} \rightarrow f(A) \wedge f(B)=\{z=5\}$ $\leftarrow$ 상수!

합치고 적용: $A \wedge B = \{x=NAC, y=NAC\} \rightarrow f(A \wedge B)=\{z=NAC\}$ $\leftarrow$ 상수 아님!

$f(A \wedge B) \neq f(A) \wedge f(B) \rightarrow$ 비분배!

반복 알고리즘은 $z=NAC$ 로 판단 (정밀도 손실, but 안전)

4-7. 해의 계층

$FP \leq MFP \leq MOP \leq \text{Perfect Solution}$

$\leftarrow$ 보수적/안전 ————— 정밀/이상적 $\rightarrow$

Perfect: 실제 실행 경로만 고려 (계산 불가능!)

MOP: CFG의 모든 가능 경로 고려

MFP: 반복 알고리즘의 결과

FP: 아무 고정점 해

분배적 $\rightarrow MFP = MOP$ (최선!)

단조적 $\rightarrow MFP \leq MOP$ (안전하지만 정밀도 $\downarrow$ 가능)

4-8. 정확성 — 경로 무시가 위험한 이유

RD에서 경로를 무시하면:

정상: $d1:x=$ $d2:x=$ \ / $u1:=x$ RD: $\{d1,d2\}$ (안전) 격자: 아래쪽 (보수적)	경로 무시: $d1:x=$ $d2:x=$ \ (무시!) $u1:=x$ RD: $\{d1\}$ (위험!) 격자: 위쪽 (unsafe!)
--	--

경로 무시 $\rightarrow$ 격자에서 위로 $\rightarrow$ 정보 손실 $\rightarrow$ 잘못된 최적화 가능!

반복 알고리즘은 경로를 절대 무시 안 함 $\rightarrow$ 항상 안전

4-9. 수렴 속도 — Reverse Post-Order

Forward 분석 $\rightarrow$ rPostOrder로 방문 ($\approx$ 위상 정렬)

정보가 한 패스에 위→아래로 쪽 전파

계산법:

- ① DFS로 빠져나올 때 번호 부여 (post-order)
- ② 뒤집기: $rPO(i) = \text{전체노드수} - \text{PostOrder}(i)$

Backward 분석 → rPostOrder의 역순

실제 프로그램 평균 반복: ~2.75회 (루프 중첩 깊이에 비례)

【예시문제】 rPostOrder 알고리즘을 아래 CFG에 적용하라.

```
entry → A → B → D → exit
      ↓
      C → D
```

【풀이】:

DFS(A에서 시작):

Visit A → Visit B → Visit D → Visit exit
PostOrder: exit=1, D=2
돌아와서 Visit C → Visit D(이미 방문)
PostOrder: C=3, B=4, A=5

$rPostOrder = 5 - \text{PostOrder}$
A=0, B=1, C=2, D=3, exit=4

방문 순서: A → B → C → D → exit (위상 정렬과 유사!)

4-10. Ch4 개념 → 컴파일러 실전 연결

Ch4는 "분석 도구의 품질 보증서"

새 최적화를 설계할 때:

- "이 반복 알고리즘이 끝나는가?" → 단조성 + 유한 높이 확인
- "결과를 믿고 최적화해도 안전한가?" → 정확성 (경로 무시 안 함) 확인
- "최적화 기회를 놓치지 않는가?" → 분배성 확인 (MFP=MOP?)
- "100만줄 코드에서 몇 시간 걸리나?" → rPostOrder + 격자 높이로 예측

Ch4 개념	컴파일러 실전 용도
격자/반순서	$\cup$ vs $\cap$ 선택 기준, 초기값(Top) 결정, 정밀도 방향
격자 높이	최대 반복 횟수 상한 (RD:정의수, LV:변수수, CP:2)
단조성	반복 종료 보장. 새 분석 설계 시 필수 확인
분배성	MFP=MOP 판별. 비분배면 더 정교한 알고리즘 필요 (예: SSA 기반 SCC)
MFP/MOP	분석 품질 등급. 놓치는 최적화 범위 파악
정확성	union 사용 근거. "보수적 방향" 선택의 이론적 배경
rPostOrder	GCC/LLVM의 실제 BB 방문 순서. 평균 ~2.75회 수렴

각 분석과 최적화의 연결:

- RD (분배적) → Live Range 구축, Register Promotion, 유도변수 추적
- LV (분배적) → Dead Code 제거, Register Allocation, LICM 안전성 확인
- Available Expr (분배적) → Global CSE, LICM
- Constant Propagation (비분배) → 상수 접기/전파 (반복으로 한계 → SSA 기반 보완)

Ch6. Control Flow Analysis and Loops

6-1. 루프의 정의

루프의 두 조건:

- ① 단일 진입점 (header): 외부에서 들어오는 입구 1개
- ② 사이클: 간선들이 최소 1개 사이클 형성

모든 사이클 ≠ 루프. 진입점 2개 이상이면 최적화용 루프 아님.

6-2. Dominator

$d \text{ dom } n$: start에서 n 으로 가는 모든 경로가 d 를 통과

데이터 흐름 분석으로 계산:

- 방향: Forward, $\Lambda=n$, 전달함수: $OUT[b]=IN[b]\cup\{b\}$
경계: $IN[entry]=\{\}$, 초기화: $OUT[b]=U(\text{전체})$
분배적 $\rightarrow$ MFP=MOP

[예시문제] 노드 7의 dominator ($pred=\{5,6\}$, $OUT[5]=\{1,2,3,5\}$, $OUT[6]=\{1,2,4,6\}$):

[풀이]: $IN[7]=\{1,2,3,5\}\cap\{1,2,4,6\}=\{1,2\}$. $OUT[7]=\{1,2\}\cup\{7\}=\{1,2,7\}$

6-3. Back Edge와 Natural Loop

Back edge: $t \rightarrow h$ 에서 $h \text{ dom } t$ (" h 가 t 를 dominate")

DFS 간선 분류: Advancing(조상 $\rightarrow$ 자손), Retreating(자손 $\rightarrow$ 조상), Cross(좌 $\rightarrow$ 우)

$\text{back edge} \subseteq \text{retreating edge}$

Reducible flow graph: $\text{retreating} = \text{back}$ (구조적 코드)

Natural loop of $t \rightarrow h$:

1. h 제거 $\rightarrow$ 2. t 에서 역방향 도달 가능 노드 $\rightarrow$ 3. 그 노드들+ h = loop

Inner loop = 다른 루프 미포함. 최적화 주 대상.

Preheader = 루프 헤더 앞 새 BB. LICM 코드 삽입점.

[예시문제] $8 \rightarrow 7$ 이 back edge인지: $7 \text{ dom } 8$? 8 의 $\text{dom}=\{1,2,7,8\}$, $7 \in \text{dom} \rightarrow \text{YES!}$

Ch7. Loop Optimizations — LICM + Strength Reduction

7-1. LICM (Loop Invariant Code Motion)

Loop invariant: 루프 안에서 값이 변하지 않는 계산
 $\rightarrow$ preheader로 이동하면 반복 횟수만큼 절감!

탐지 (RD 사용, 반복):

$A=B+C$ 가 invariant $\Leftrightarrow$

B 의 모든 reaching defs가 루프 밖 or

루프 안 def 1개이고 이미 invariant

(C도 동일) $\rightarrow$ 변화 없을 때까지 반복

Code Motion 3조건:

- (1) BB가 루프 모든 exit을 dominate
- (2) 같은 변수의 다른 def가 루프 안에 없음
- (3) A의 모든 use가 해당 def에 dominate됨

[예시문제] invariant $A=B+C$ 를 preheader로 옮겨도 안전한 조건은?

[풀이]: (1) $A=B+C$ 가 있는 BB가 루프 exit을 dominate (2) 루프 안에 A의 다른 정의 없음 (3) A를 사용하는 모든 곳이 이 def에 dominate됨. 셋 다 충족해야 이동 가능.

7-2. Strength Reduction

BIV: $X = X \pm c$ (루프 내 유일한 정의)
 IV: $A = c1 \times BIV + c2$

- SR: 1. 새 A' 생성
 2. Preheader: $A' = c1 \times B + c2$
 3. B 변경 뒤: $A' = A' + x \times c1$ (곱셈→덧셈!)
 4. $A = A'$

LFTR: 루프 조건을 IV 기반으로 교체 → BIV 제거
 Non-trivial BIV: 정의 여러 개여도 각 def 뒤에 IV 업데이트 추가

[예시문제] for($i=0; i<100; i++$) { $t1=4*i; t2=\&A+t1; *t2=0; \}$ 에 SR 적용.

[풀이]:

BIV i , IV $t1=4i, t2=4i+\&A$
 preheader: $t2'=\&A, t3=\&A+400$
 loop: if($t2' \geq t3$) goto L1; $*t2'=0; t2'+=4; goto loop$
 → 곱셈 제거, i 제거, 루프 3명령어

Practice Problem 2: Anticipation Analysis + True Value Analysis

PP2-1. Anticipation Analysis 전체 풀이

프레임워크: Backward, meet=intersection, 분배적
 전달: $IN[b] = GEN[b] \cup (OUT[b] - KILL[b])$
 합류: $OUT[b] = \text{intersection } IN[succ]$
 경계: $OUT[exit]=\{\}$. 초기화: $IN[b]=U(\text{전체})$

명령어 " $w = x+y$ " (Backward):
 GEN: $\{x+y\}$ (계산되므로 위에서 anticipated)
 KILL: $\{w\text{포함 식}\}$ (w 재정의하면 위의 w ? 무효)

" $a = a+b$ ": $GEN=\{a+b\}, KILL=\{a\text{포함}\}$. GEN이 KILL보다 우선 → $a+b$ 살아남음

BB합성(Bwd): $GEN_BB = gen_위 \cup (gen_아래 - kill_위)$
 $KILL_BB = kill_위 \cup kill_아래$

[예시문제] CFG: $BB0 \rightarrow \{BB1, BB2\} \rightarrow BB3$. $BB1: b=a+b, d=b+c$. $BB2: e=a+b, g=a+c$. $BB3: f=b+c, a=a+c$. 표현식: $a+b, a+c, b+c$. Anticipated를 구하라.

[풀이]:

GEN/KILL (Backward 합성):
 $BB0: GEN=\{\}, KILL=\{\}$

BB1 (d=b+c 먼저, b=a+b 나중):
 d=b+c: gen={b+c}, kill={}
 b=a+b: gen={a+b}, kill={a+b,b+c}
 합성: GEN={a+b} union ({b+c}-{a+b,b+c})={a+b}
 KILL={a+b,b+c}

BB2 (g=a+c 먼저, e=a+b 나중):
 GEN={a+b,a+c}, KILL={}

BB3 (a=a+c 먼저, f=b+c 나중):
 a=a+c: gen={a+c}, kill={a+b,a+c}
 f=b+c: gen={b+c}, kill={}
 합성: GEN={b+c,a+c}, KILL={a+b,a+c}

반복(Bwd, 초기 IN=U):
 BB3: OUT={} -> IN={b+c,a+c}
 BB1: OUT={b+c,a+c} -> IN={a+b} union ({b+c,a+c}-{a+b,b+c})=
 {a+b,a+c}
 BB2: OUT={b+c,a+c} -> IN={a+b,a+c} union {b+c,a+c}={a+b,a+c,b+c}
 BB0: OUT=IN[BB1] inter IN[BB2]={a+b,a+c} inter {a+b,a+c,b+c}=
 {a+b,a+c}
 IN={a+b,a+c}
 2회차: 변화 없음 -> 수렴!

최종: BB0:IN/OUT={a+b,a+c}. BB1:IN={a+b,a+c}. BB2:IN={a+b,a+c,b+c}.
 BB3:IN={b+c,a+c}

해석: BB0 OUT에 b+c 없음 = BB2 경로에서 b+c 미계산 -> inter 탈락
 BB1 IN에 b+c 없음 = b=a+b로 b 재정의 -> 위의 b+c가 아래의 b+c와 다
 른 값 -> KILL

PP2-2. True Value Analysis

Boolean 변수 (a=TRUE/FALSE/b/NOT b)
 각 지점에서 possibly TRUE인지

도메인(1변수): {}, {T}, {F}, {T,F}
 격자 (meet=union):
 {T,F}=Bottom, {}=Top
 (meet=union에서 큰 집합=아래)

Forward, meet=union
 경계: IN[entry]={}
 초기화: OUT[b]={} (=Top)

전달함수 (변수 x에 대해):
 x 미대입: OUT(x) = IN(x)
 x=TRUE: OUT(x) = {T}
 x=FALSE: OUT(x) = {F}
 x=y: OUT(x) = IN(y)
 x=NOT y: OUT(x) = flip(IN(y))
 flip: {}->{} {T}->{F} {F}->{T} {T,F}->{T,F}

단조적 확인 (a=NOT b):
 IN1={T,F}, IN2={T}: {T,F} <= {T} (큰집합이 아래)
 OUT1=flip({T,F})={T,F}, OUT2=flip({T})={F}
 {T,F} <= {F} -> YES!

IN1={F}, IN2={}: {F} <= {}
 OUT1=flip({F})={T}, OUT2=flip({})={}
 {T} <= {} -> YES!

수렴: 높이=2 + 단조 -> 반드시 수렴!

Ch11. Static Single Assignment (SSA) Form

11-1. SSA 정의와 장점

SSA = 각 변수가 프로그램 텍스트에서 딱 한 번만 정의되는 형태

"Static": 텍스트상 1번. 루프 안이면 런타임에 여러번 실행될 수 있음

변환 전:	변환 후 (SSA):
a = x + y	a1 = x + y
b = a - 1	b1 = a1 - 1
a = y + b ← a 재정의	a2 = y + b1 ← 새 이름!
b = x * 4 ← b 재정의	b2 = x * 4
a = a + b	a3 = a2 + b2

장점:

- 1) use → def가 이름만으로 즉시 연결 (RD 분석 불필요!)
- 2) AE에서 KILL = {} (변수가 재정의 안 되므로)
- 3) Def-Use 체인: $O(N*M) \rightarrow O(N+M)$
- 4) 상수 전파, CSE 등이 극적으로 단순화

11-2. ϕ -function — 합류점에서 값 선택

분기 후 합류점에서 "어느 경로로 왔느냐"에 따라 다른 변수를 선택:

```
a1 = 0
if (b1 < 4)
    a2 = b1          ← then 경로
a3 = phi(a2, a1)    ← then이면 a2, else면 a1
c1 = a3 + b1
```

phi = 표기법(notational fiction). 실제 실행 안 됨!
경로에 따라 값을 선택하는 분석용 표기.

11-3. Dominance Frontier — phi를 어디에 삽입?

$DF(x) = \{w \mid x \text{가 } w \text{의 어떤 pred를 dominate \& } x \text{가 } w \text{를 strictly-dominate 안 함}\}$

직관: x의 지배 영역 경계. "x의 영향력이 끝나고 다른 경로와 만나는 지점"

phi 삽입 규칙: 변수 a를 정의하는 BB x에 대해, $DF(x)$ 의 각 노드에 phi 삽입
이름 변경: dominator tree를 BFS 방문, 각 use를 가장 가까운 def 이름으로 교체

[예시문제] 아래를 SSA로 변환하라.

```
b=M[x]; a=0; if(b<4) a=b; c=a+b;
```

[풀이]:

```
b1=M[x0]; a1=0; if(b1<4) a2=b1;
a3=phi(a2,a1); c1=a3+b1
→ a3은 then이면 a2(=b1), else면 a1(=0)
```

11-4. Back Transformation — SSA를 일반 형태로

phi(x1, x2)를 각 predecessor 끝에 copy로 변환:

phi(x1, x2) → pred1 끝에 "y = x1", pred2 끝에 "y = x2"

★ Lost Copy Problem:

Copy propagation 후 back transform하면 순서 문제 발생 가능!

해결: Edge-Split SSA Form

간선 a→b에서 a가 succ 2개+, b가 pred 2개+이면

빈 노드를 사이에 삽입 → copy 충돌 방지

11-5. SSA는 업계 표준

LLVM: 내부 IR이 SSA. DCE/CSE/LICM/상수전파 모두 SSA 기반

Android Dalvik VM, Java HotSpot VM, V8 JS Engine: SSA 사용

Ch8. Register Allocation — Graph Coloring

8-1. 문제와 용어

문제: 의사 레지스터(수백~수천) → 물리 레지스터(16~32개) 배정

Allocation: 레지스터에 유지 결정. Assignment: 어떤 레지스터에. Spilling: 메모리로.

8-2. 간섭 그래프와 컬러링

간섭: 두 변수가 어떤 지점에서 동시에 live → 다른 레지스터 필요

간섭 그래프: 노드=live range, 간선=동시 live 쌍

레지스터 할당 = n-coloring (n=물리 레지스터 수). NP-complete → 휴리스틱

Live range = 정의 d가 도달(RD) AND 변수가 live(LV)인 지점의 집합

겹치는 live range는 merge → 노드가 정밀해짐 → 컬러링 쉬워짐

8-3. Chaitin's Algorithm

핵심: degree < n인 노드는 항상 컬러링 가능!

[Simplify] degree < n 노드를 스택에 push+제거 (이웃 degree 감소)

[Assign] 스택에서 pop하면서 역순으로 색 배정 (반드시 가능!)

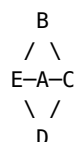
[Stuck] 모든 degree ≥ n → spill 필요

Spill 비용/이득: cost=사용횟수×루프가중치, benefit=degree

→ cost/degree 낮은 노드를 우선 spill

→ spill 후 load/store 삽입 → 그래프 재구축 → 반복

[예시문제] n=3, 그래프 B-E, B-A, B-C, E-A, E-D, A-C, A-D, C-D를 컬러링하라.



모든 노드 degree=3 또는 4 (A=4, B=C=D=E=3)

[풀이 — Chaitin]: 모든 degree ≥ 3 → stuck! → spill 필요 (비관적)

[풀이 — Briggs Optimistic]:

Simplify: degree < 3 없음 → D를 스택 push(spill 미확정), 제거

D 제거 후: A(deg2), E(deg2) → degree < 3 발생!

E(deg2)→push. A(deg2→1)→push. B(deg1)→push. C(deg0)→push.

스택: C,B,A,E,D

Assign: C=R1, B=R2, A=R3, E=R1, D=R2(이웃:E=R1,A=R3,C=R1→R2가능!)

→ spill 없이 3색 성공! Briggs가 Chaitin보다 우수한 예.

8-4. Optimistic Coloring (Briggs)

Chaitin: degree $\geq$ n → 즉시 spill 결정 (비관적)

Briggs: degree $\geq$ n → 스택에 넣고 나중에 색칠 시도 (낙관적)

Assign에서 실제로 색 없으면 그때 spill

→ Chaitin보다 같거나 적은 spill!

예: 다이아몬드 그래프(n=2) → Chaitin은 spill, Briggs는 2색 가능

8-5. Live Range Splitting

Case 1: 거의 작은 구간 분리 → 번갈아 같은 레지스터 사용 (store/load 경계)

Case 2: copy 삽입으로 다른 레지스터 배정 → spill 회피

8-6. Copy Coalescing

COPY x,y에서 x,y 비간섭 → 합쳐서 같은 레지스터 → COPY 삭제

전략: Aggressive(무조건) / Conservative(degree 안 늘면만) /

Iterated(simplify↔coalesce 반복) / Optimistic(합치되 spill시 분리)

8-7. Caller/Callee-Save + Pre-colored

Caller-save: 호출자 저장. 호출 경계 안 넘는 변수에 배정.

Callee-save: 피호출자 저장(entry/exit 1번). 호출 넘는 변수에 배정.

Pre-colored: SP,FP,GP,인자,리턴값 등 미리 고정된 물리 레지스터.

Ch9. Instruction Scheduling

9-1. 스케줄링이란

독립적 명령어를 재배치하여 파이프라인 활용 극대화, stall 제거

Parallel group: 매 사이클 독립 명령어를 묶어 동시 실행

9-2. 두 가지 제약

(1) 자원 제약: 매 사이클 실행 유닛 수 제한 (예: ALU 1 + MEM 1)

(2) 의존성 제약:

True (RAW): write→read. 진짜. delay $>$ 0

Anti (WAR): read→write. 이름 충돌. delay=0 (같은 사이클 병렬 가능)

Output (WAW): write→write. 이름 충돌.

Anti/Output는 renaming으로 해소 가능

9-3. 의존성 극복

제어 의존성 → Speculative code motion

3조건: 정확성(새변수), 예외무발생, 영구변경안함(store불가)

Anti/Output → On-demand renaming (새 이름 도입)

메모리 → Memory disambiguation (No/Yes/Maybe)

9-4. List Scheduling

1. DAG 구성: 노드=명령어, 간선=의존성(delay)
 2. READY = 선행자 0인 노드
 3. READY에서 우선순위 최고 선택 -> 가장 빠른 슬롯 배정
 4. 우선순위: critical path > delay > source order
- NP-complete -> 휴리스틱 (backtrack 안 함)

[예시문제] i1:LD r2=9(r1), i2:ST r4,0(r3), i3:ADD r4=r4,r3, i4:ADD r1=r5,r4, i5:ADD r6=r2,r4 스케줄 (ALU1+MEM1, LD=2cyc).

[풀이]: DAG: i1->(2)i5, i2->(0)i3, i3->(1){i4,i5}, i1->(0)i4. Cyc1: [MEM:i1, ALU:i3]. Cyc2:[MEM:i2, ALU:i4]. Cyc3:[ALU:i5].

9-5. 방향과 RA 상호작용

Top-down(AEAP): 빠른 시작, 레지스터 수명 길어짐
 Bottom-up(ALAP): 수명 짧음, expression tree 유리
 RA 순서: pre-RA(자유도 높) -> RA -> post-RA(spill 반영)
 Integrated: 레지스터 여유시 병렬성, 부족시 수명축소 우선

Ch10. Software Pipelining

10-1. 개념과 동기

루프의 서로 다른 반복을 겹쳐 실행 (CPU 파이프라인과 같은 개념을 SW로)

예: $A[i]=A[i]*b+c$, MEM1+ALU1 머신
 한 반복: Read(1cyc)→Mul(2cyc)→Add(2cyc)→Write(1cyc) = 6cyc/iter
 최적(자원 기준): MEM 2개+ALU 2개 → 2cyc/iter 가능

SW 파이프라이닝 Kernel:
 cyc A: [Add(i)] [Read(i+2)] ← MEM+ALU 동시!
 cyc B: [Write(i)] [Mul(i+1)] ← MEM+ALU 동시!
 → $II=2cyc/iter$ = 최적!

10-2. 파이프라인 루프 구조

Prolog: 파이프라인 채움 (점진적 겹침 시작)
 Kernel: 정상 상태 (매 II사이클마다 1반복 완료, 반복)
 Epilog: 파이프라인 비움 (마지막 반복들 완료)

II (Initiation Interval) = Kernel 사이클 길이 = 반복 시작 간격
 II 가 작을수록 성능 좋음

10-3. Unrolling vs SW Pipelining

Unrolling: 실행시간 = $4+2u$ (u 번 unroll). 코드 u 배 증가. 효율90%→ $u=18$
 SW Pipeline: $II=2$ 최적 도달. 코드 2~3배만 증가. Unrolling보다 우수.

10-4. Modulo Scheduling

$MII = \max(RMII, PMII)$

$RMII = \max(r_i/R_i)$: 각 자원별 (사용량/CPU보유량)의 최대
 예: ALU4개/CPU2개=2, MEM3개/CPU1개=3 → $RMII=3$

$PMII = \max(\sum(d)/\sum(p))$: 의존성 그래프 사이클별
 간선 $\langle p, d \rangle$: p =반복간격, d =지연. 사이클의 d 합/ p 합

알고리즘:

1. $II=MII$ 로 시작
2. Modulo Reservation Table에 배정 시도
3. 성공→확정, 실패→ $II++$ 로 재시도

[예시문제] 2-ALU+1-MEM 머신, 한 반복에 ALU 4개+MEM 2개. RMII는?

[풀이]: $RMII(ALU)=4/2=2$, $RMII(MEM)=2/1=2 \rightarrow RMII=2$

10-5. Modulo Variable Expansion + Rotating Register

수명>II인 변수: 다음 반복에서 덮어쓰기 위험

→ 여러 레지스터 할당 + 커널 unroll

또는: Rotating Register (Itanium) – HW가 자동 시프트, unroll 불필요

10-6. EPS (Enhanced Pipeline Scheduling)

MS와 다른 접근: 원래 루프 유지, DAG 스케줄링 반복 적용

코드가 백엣지 넘어 이동 → 파이프라이닝 효과

장점: 임의 제어흐름/비상수 반복수/outer loop/variable II 처리 가능